OOPs Concepts in Python

What is OOP?

Object-Oriented Programming (OOP) is a programming paradigm based on the concept of objects (data + methods). It is used for reusability, modularity, data hiding, and real-world modeling.

Basic OOP Terminologies

1. Class → Blueprint for creating objects.
2. Object → Instance of a class.
3. Constructor (__init__) → Special method called when an object is created.
4. self → Refers to the current object.

Types of Constructors in Python

1. Default Constructor → Does not accept arguments, initializes default values.
2. Parameterized Constructor → Accepts arguments to initialize values.
3. Copy Constructor → Creates a new object as a copy of an existing object.

Four Pillars of OOP in Python

1. Encapsulation → Wrapping data & methods into a single unit (class).
2. Inheritance → One class can acquire properties of another.
3. Polymorphism → One name, many forms (method overriding, operator overloading).
4. Abstraction → Hiding implementation details, showing only essential features.

Method Types

1. Instance Method → Works with object (self).
2. Class Method → Works with class (@classmethod, cls).
3. Static Method → Independent of class & object (@staticmethod).

Access Modifiers

1. Public: var
2. Protected: _var
3. Private: __var

Duck Typing in Python

If an object behaves like a type, it can be treated as that type.

Advantages of OOP

✔ Code reusability
✔ Data security

✔ Modularity
✔ Scalability
✔ Easy to debug and maintain

OOPs Example: Basic Arithmetic Functions

We can implement addition, subtraction, multiplication, division, and modulus using OOP concepts.

Example:

```
class Calculator:
def __init__(self(self, a, b):
self.a = a
self.b = b

def addition(self):
return self.a + self.b

def subtraction(self):
return self.a - self.b

def multiplication(self):
return self.a * self.b

def division(self):
return self.a / self.b

def modulus(self):
return self.a % self.b

obj = Calculator(10, 3)
print('Addition:', obj.addition())
print('Subtraction:', obj.subtraction())
print('Multiplication:', obj.multiplication())
print('Division:', obj.division())
print('Modulus:', obj.modulus())
```

Decorators in Python

Decorators are a powerful way to extend the functionality of functions or methods without modifying their source code. They are implemented as functions that take another function as an argument and return a new function that wraps the original function.Decorators are often used for logging, timing, access control, and more.

Magic Methods in Python

Magic methods, also known as dunder methods (due to their double underscores), are special methods in Python classes that allow you to define how objects of your class interact with built-in functions and operators. Examples include `__init__` for initialization, `__str__` for string representation, `__add__` for the addition operator, and many more. Using magic methods allows your objects to behave like built-in types, making your code more Pythonic and readable.

Inheritance in Python

Inheritance is a mechanism that allows a new class to inherit properties and behaviors from an existing class. The existing class is called the base class or parent class, and the new class is called the derived class or child class. Inheritance promotes code reusability and establishes a hierarchical relationship between classes.

Generators in Python

Generators are a simple way to create iterators. They are functions that use the `yield` keyword to produce a sequence of values. Unlike normal functions that return a value and exit, generators can yield multiple values over time, pausing and resuming their execution state.

Collections in Python

Python's `collections` module provides specialized container datatypes that are alternatives to Python's general-purpose built-in containers like `list`, `dict`, `tuple`, and `set`. Examples include `Counter` for counting hashable objects, `deque` for fast appends and pops from both ends, `namedtuple` for creating tuple subclasses with named fields, and `defaultdict` for dictionaries with default values for missing keys.

File Handling in Python

File handling in Python involves reading from and writing to files. Python provides built-in functions like `open()`, `read()`, `write()`, and `close()` to interact with files. Proper file handling is crucial for persistent data storage and retrieval.

Exception Handling in Python

Exception handling is a mechanism to deal with errors that occur during the execution of a program. Python uses `try`, `except`, `else`, and `finally` blocks to catch and handle exceptions gracefully, preventing the program from crashing.

Methods in Python Classes

Methods are functions defined inside a class that describe the behaviors or actions of objects created from that class. Methods can access and modify the object's attributes. There are different types of methods, including instance methods, class methods, and static methods, each with its own use case.

Polymorphism in Python

Polymorphism is a concept in object-oriented programming that refers to the ability of different objects to respond to the same method call in their own way. In Python, polymorphism is often achieved through method overriding (defining a method in a subclass with the same name as a method in the superclass) and operator overloading (defining how operators like `+` or `*` should behave for custom objects).